

INT 161

Basic Backend Development

Week-02

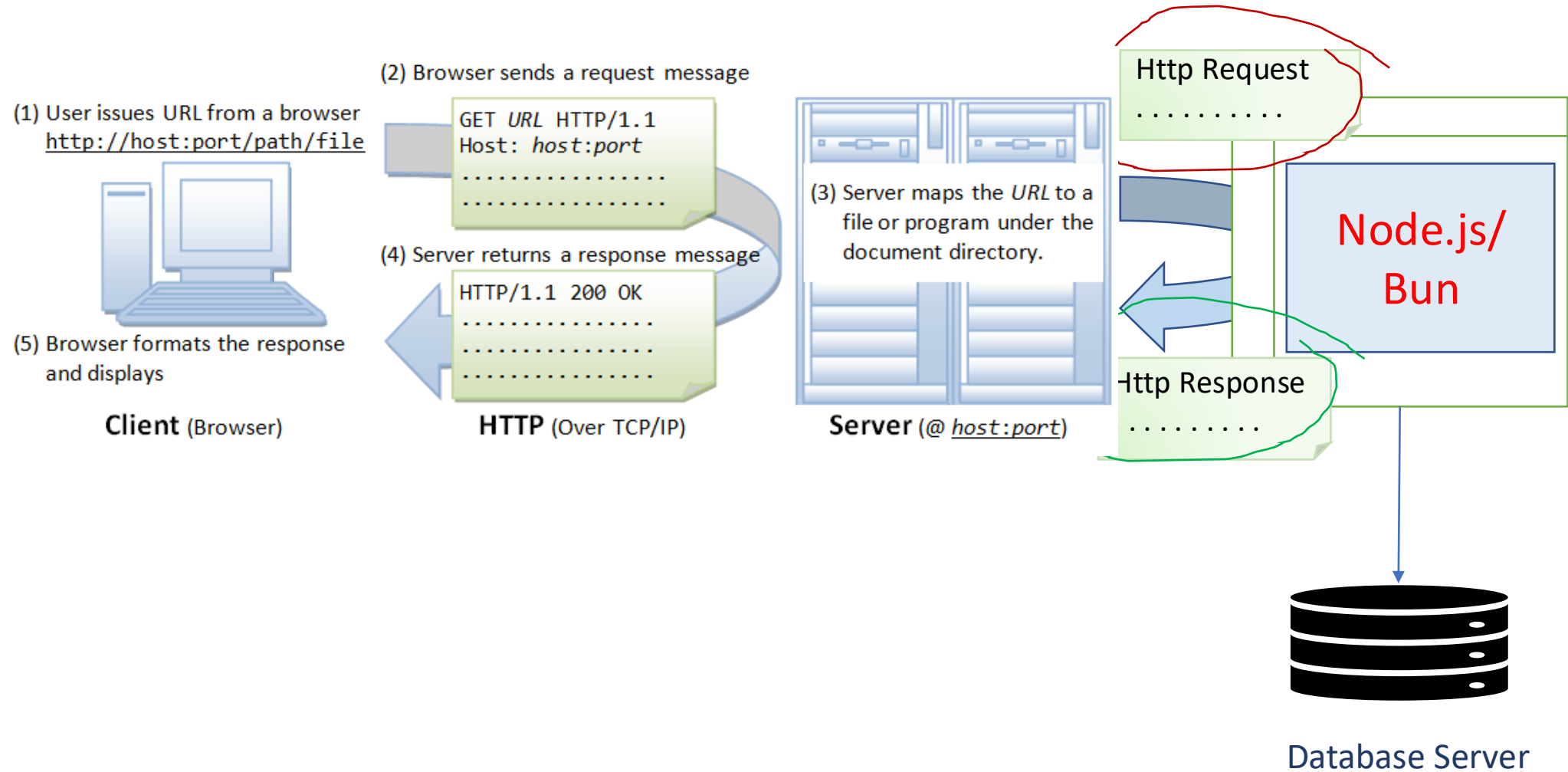


HTTP Programming Basics

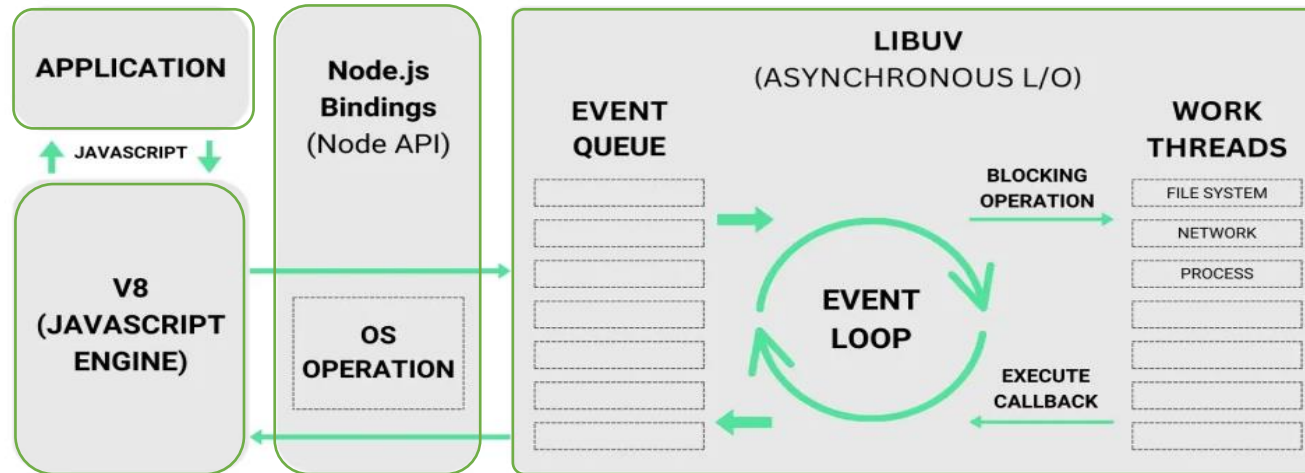
Unit Objectives

- After completing this unit, you should be able to:
 - Explain the HTTP protocol for Web Programming
 - Created HTTP Server using Node.js
 - Manage Request & Response objects
 - Read Request parameter/Request body and path variable
 - Created Express.js project

Web Application



Node.js Architecture



- Clients send requests to the webserver to interact with the web application. Requests can be non-blocking or blocking e.g. Querying the data , updating the data or deleting the data .
- Node.js retrieves the incoming requests and adds those requests to the Event Queue.
- The requests are then passed one-by-one through the Event Loop. It checks if the requests are simple enough to not require any external resources.
- Event Loop processes simple requests (non-blocking operations), such as I/O Polling, and returns the responses to the corresponding clients.

Node.js Core Built-in Modules

- Here are some of the most commonly used ones:
 - fs - File system operations
 - http - HTTP server and client
 - path - File path utilities
 - os - Operating system utilities
 - events - Event handling
 - util - Utility functions
 - stream - Stream handling
 - crypto - Cryptographic functions
 - url - URL parsing
 - querystring - URL query string handling

https://www.w3schools.com/nodejs/nodejs_modules.asp

<https://www.geeksforgeeks.org/node-js/node-js-core-modules/>

Node.js HTTP Module

<https://www.geeksforgeeks.org/node-js/node-js-http-module/>

- The **built-in HTTP module** in Node.js, used to create HTTP servers and make HTTP client requests - basically, lets Node.js talk over the web without needing any external library.
- Key Features
 - Create HTTP servers to handle requests and send responses
 - Make HTTP requests to other servers
 - Handle different HTTP methods (GET, POST, PUT, DELETE, PATCH, etc.)
 - Work with request and response headers
 - Handle streaming data for large payloads
- Importing the

```
// Using ES Modules (Recommended)  
import * as http from 'node:http';
```

```
// Using CommonJS  
const http = require('node:http');
```

https://www.w3schools.com/nodejs/nodejs_http.asp

Which one should you use?

Use ES Modules (ESM) for **all new projects**. It is the official standard for JavaScript moving forward.

Use CommonJS (CJS) **only if you work on an older**, legacy Node.js codebase where migrating would break existing dependencies.

CJS vs ESM

<https://www.geeksforgeeks.org/node-js/node-js-modules/>

- CommonJS uses `require()` and loads **synchronously**, making it the traditional Node.js standard.
- ES Modules use `import/export`, support **asynchronous** loading, and are the official modern web standard.
- Key Differences
 - Syntax: CJS uses `require()` and `module.exports`; ESM uses `import` and `export`.
 - Loading: CJS loads files synchronously at runtime; ESM is parsed statically and supports asynchronous loading.
 - Environment: CJS was built for server-side Node.js; ESM works natively in both browsers and Node.js.
 - Optimization: ESM allows bundlers to do "tree-shaking" to remove unused code; CJS does not support this well.
 - Variables: CJS provides local file variables `__dirname` and `__filename`; ESM uses `import.meta.url`.

https://www.w3schools.com/nodejs/nodejs_modules_esm.asp

HTTP Module Core Classes (1/2)

- The node:http module is built around four primary classes:
 1. **http.Server** - This object represents an HTTP server instance.
 - **server.listen(port, callback)**: Starts the server listening for connections on a specified port.
 - **server.close(callback)**: Stops the server from accepting new connections.
 - **Event: 'request'**: Emitted every time there is a request. Passes an IncomingMessage and a ServerResponse object.
 2. **http.ServerResponse** - Created internally by the server to construct and send the response back to the client.
 - **response.writeHead(statusCode, headers)**: Sends a response header to the request.
 - **response.setHeader(name, value)**: Sets a single header value for implicit headers.
 - **response.write(chunk)**: Sends a chunk of the response body.
 - **response.end(data)**: Signals to the server that all response headers and body have been sent.

HTTP Module Core Classes (2/2)

3. `http.IncomingMessage` - Created by `http.Server` or `http.ClientRequest` to read incoming request data or client responses. It implements the `ReadableStream` interface.

- **`request.method`**: The request method as a string (e.g., 'GET', 'POST').
- **`request.url`**: The request URL string.
- **`request.headers`**: An object containing the parsed incoming request headers

4. `http.ClientRequest` - Created internally when calling `http.request()`. It represents an in-progress client-side HTTP request.

- **`request.write(chunk)`**: Sends a body chunk for a POST/PUT
- **`request.request.end()`**: Finishes sending the request.
- **Event: 'response'**: Emitted when a response is received from the server.

Http Server - Request and Response

http-server.js

```
import * as http from 'node:http';

const listener = function (request, response) {
  const { url, method } = request;
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(`You made a ${method} request to ${url}`); //template literal
};

const server = http.createServer(listener);
server.listen(3000, () => {
  console.log('Server running at http://127.0.0.1:3000/');
});
server.on('request', (req, res) => {
  console.log('Request received:', req.method, req.url);
});
```

```
curl -i -X PUT http://localhost:3000/user/5 -d '{"name": "John Doe", "role": "Admin"}'
```

HTTP Methods

- **Method Purpose Example**

- GET Retrieve data from resource /subjects /students
- POST Create new resource /subjects /students (with data)
- PUT Update existing data /subjects/{id} /students /{id} (with data)
- DELETE Remove a resource /subjects/{id} /students /{id}

Parsing URLs with the URL Module

- The Node.js url module provides utilities for URL resolution, parsing, and manipulation.
- Modern Node.js environments primarily use the WHATWG (Web Hypertext Application Technology Working Group) URL API, which makes the URL and URLSearchParams classes globally available without needing a require() statement.

http://localhost:3000/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dec

```
const urlObj = new URL(request.url, `http://${request.headers.host}`);  
const params = urlObj.searchParams;  
  
const name = params.get('name');  
const subjects = params.getAll('fav_subj');
```

https://www.w3schools.com/nodejs/nodejs_http.asp

Request parameters

```
const listener = function (request, response) {  
  response.writeHead(200, { 'Content-Type': 'text/plain' });  
  const { url, method } = request;  
  const urlObj = new URL(request.url, `http://${request.headers.host}`);  
  const params = urlObj.searchParams;  
  const name = params.get('name');  
  const subjects = params.getAll('fav_subj');  
  response.write(`Name = ${name} Favorite Subjects: ${subjects}\n`);  
  response.end(`You made a ${method} request to ${url}`); //template literal  
};
```

http-server.js

http://localhost:3000/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dec

Request parameters (2)

```
const listener = function (request, response) {
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  const { url, method } = request;
  const urlObj = new URL(request.url, `http://${request.headers.host}`);
  const params = urlObj.searchParams;
  const name = params.get('name');
  const subjects = params.getAll('fav_subj');
  response.write('Name: ' + name + '\n');
  response.write('Favorite Subjects:\n');
  subjects.forEach(function(subject, index) {
    response.write('      ' + (index+1) + ', ' + subject + '\n\n');
  });
  response.end(`You made a ${method} request to ${url}`); //template literal
};
```

http://localhost:3000/user?name=Somchai&fav_subj=Back-end Dev&fav_subj=Front-end Dec

Request parameters (2)

http-server-path.js

```
const listener = function (request, response) {
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  const { url, method } = request;
  const urlObj = new URL(request.url, `http://${request.headers.host}`);
  const pathSegments = urlObj.pathname.split('/').filter(Boolean);
  response.write('Path Segments:\n');
  pathSegments.forEach(function(pathSegment, index) {
    response.write('      ' + (index+1) + ', ' + pathSegment + '\n\n');
  });
  response.end(`urlObj.pathname: ${urlObj.pathname}`); //template literal
  console.log('path', urlObj.pathname);
};
```

pathname.split('/') → gives an array like ["", "users", "123"] for /users/123

filter(Boolean) → runs Boolean(item) on each element:

filter() keeps only elements where the callback returns true, so empty strings are removed.

Exercise - Multiplication table

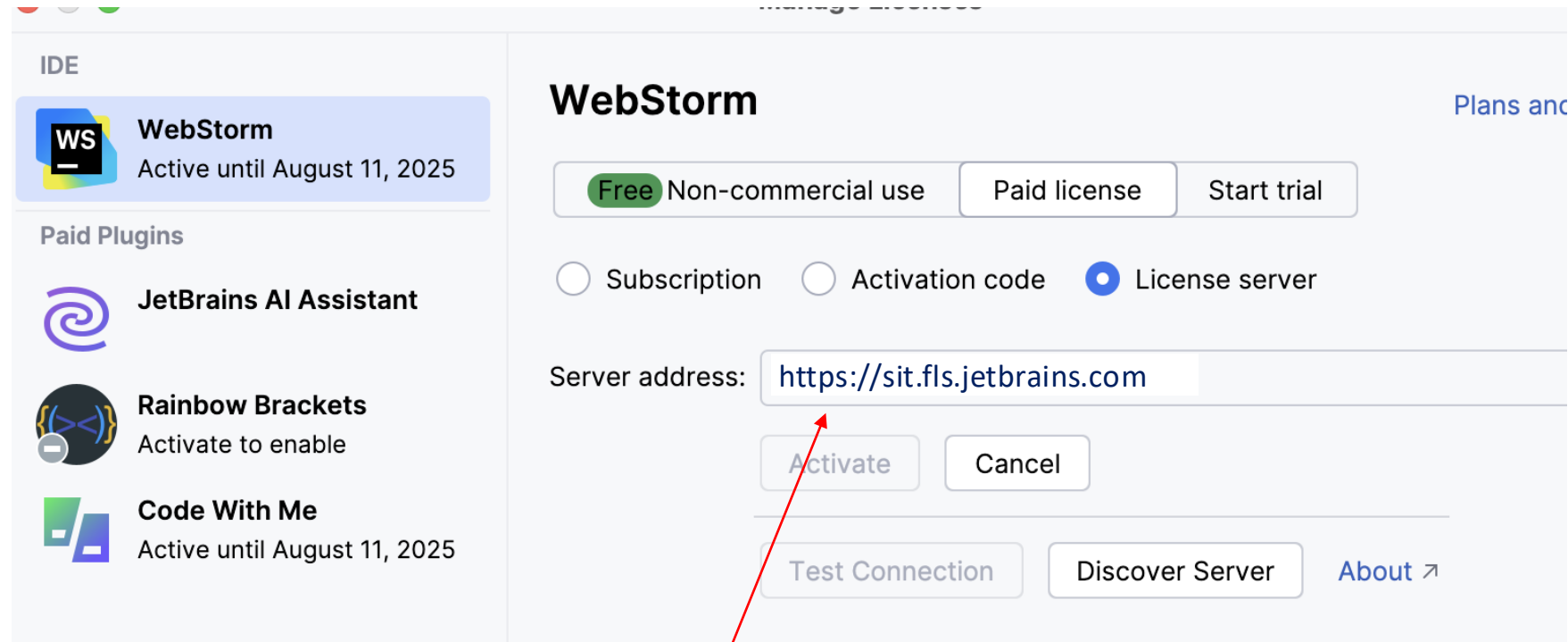
- Receive the request parameter 'number'
- returns a multiplication table for that number
- Example of response body

localhost:3000/user?number=8

Multiplication number for 8

8	x	1	=	8
8	x	2	=	16
8	x	3	=	24
8	x	4	=	32
8	x	5	=	40
8	x	6	=	48
8	x	7	=	56
8	x	8	=	64
8	x	9	=	72
8	x	10	=	80
8	x	11	=	88
8	x	12	=	96

JetBrains License



`https://sit.fls.jetbrains.com`

*** Register JetBrains account with SIT email first ! ***

Test WebStorm

- Create New Project, test

